

1. AIM

To design and implement a Bank Account Management System in Java using object-oriented programming concepts such as attributes, methods, constructors, object creation, inheritance, method overriding, deposits, withdrawals, interest calculation, and fund transfers.

2. DESCRIPTION

The program creates a base class named Account containing account number, account holder name, balance, and account type. It provides methods for depositing money, withdrawing money, transferring funds, and displaying account details.

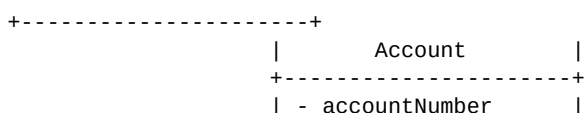
SavingsAccount inherits from Account and adds an interestRate attribute with a calculateInterest() method. CurrentAccount inherits from Account and adds an overdraftLimit attribute. It overrides the withdraw() method so that withdrawal is permitted up to the available balance plus the overdraft limit.

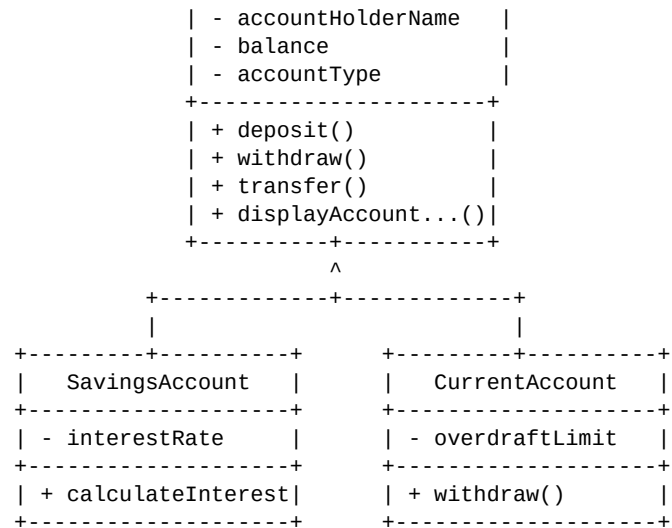
Objects of SavingsAccount and CurrentAccount are created and used to perform deposits, withdrawals, interest calculation, and fund transfers. Account details are displayed before and after transactions.

3. ALGORITHM

1. Start the program.
2. Create the base class Account.
3. Define accountNumber, accountHolderName, balance, and accountType attributes.
4. Define a constructor to initialize the account details.
5. Define deposit(), withdraw(), transfer(), and displayAccountDetails() methods.
6. Create SavingsAccount by inheriting Account.
7. Add interestRate and define calculateInterest() to calculate and add interest to the balance.
8. Create CurrentAccount by inheriting Account.
9. Add overdraftLimit and override withdraw() to allow withdrawal within the overdraft limit.
10. Create SavingsAccount and CurrentAccount objects.
11. Display the initial account details.
12. Perform deposits, withdrawals, and interest calculation.
13. Transfer money from the SavingsAccount to the CurrentAccount.
14. Display the updated account details and final balances.
15. Stop the program.

4. UML DIAGRAM





5. SOURCE CODE

```

class Account {
    int accountNumber;
    String accountHolderName;
    double balance;
    String accountType;

    // Constructor initializes all account details
    Account(int accountNumber, String accountHolderName,
            double balance, String accountType) {
        this.accountNumber = accountNumber; // Assign account number
        this.accountHolderName = accountHolderName;
        this.balance = balance; // Set opening balance
        this.accountType = accountType;
    }

    void deposit(double amount) {
        balance = balance + amount; // Add amount to balance
        System.out.println("Amount deposited successfully.");
    }

    void withdraw(double amount) {
        if (amount <= balance) { // Check available balance
            balance = balance - amount; // Deduct withdrawal amount
            System.out.println("Amount withdrawn successfully.");
        } else {
            System.out.println("Insufficient balance.");
        }
    }

    void transfer(Account receiver, double amount) {
        if (amount <= balance) { // Check balance before transfer
            balance = balance - amount; // Deduct from sender
            receiver.balance = receiver.balance + amount; // Add to receiver
            System.out.println("Amount transferred successfully.");
        } else {
            System.out.println("Insufficient balance for transfer.");
        }
    }

    void displayAccountDetails() {
        System.out.println("Account Number : " + accountNumber);
        System.out.println("Account Holder : " + accountHolderName);
        System.out.println("Account Type : " + accountType);
        System.out.println("Balance : " + balance); // Display current balance
    }
}

```

```

class SavingsAccount extends Account {
    double interestRate;

    SavingsAccount(int accountNumber, String accountHolderName,
                    double balance, String accountType,
                    double interestRate) {
        super(accountNumber, accountHolderName, balance, accountType); // Call parent constructor
        this.interestRate = interestRate; // Set interest rate
    }

    void calculateInterest() {
        double interest = balance * interestRate / 100; // Calculate interest
        balance = balance + interest; // Add interest to balance
        System.out.println("Interest Rate : " + interestRate + "%");
        System.out.println("Interest Added: " + interest);
    }
}

class CurrentAccount extends Account {
    double overdraftLimit;

    CurrentAccount(int accountNumber, String accountHolderName,
                    double balance, String accountType,
                    double overdraftLimit) {
        super(accountNumber, accountHolderName, balance, accountType); // Initialize parent class
        this.overdraftLimit = overdraftLimit; // Set overdraft limit
    }

    @Override
    void withdraw(double amount) {
        if (amount <= balance + overdraftLimit) { // Allow withdrawal within overdraft limit
            balance = balance - amount;
            System.out.println("Amount withdrawn successfully.");
        } else {
            System.out.println("Overdraft limit exceeded.");
        }
    }
}

public class BankAccountManagement {
    public static void main(String[] args) {
        SavingsAccount savings =
            new SavingsAccount(101, "Meghana",
                               10000, "Savings", 5); // Create SavingsAccount object

        CurrentAccount current =
            new CurrentAccount(102, "Rahul",
                               5000, "Current", 3000); // Create CurrentAccount object

        System.out.println("===== SAVINGS ACCOUNT =====");
        savings.displayAccountDetails();
        System.out.println("\nDepositing Rs.2000");
        savings.deposit(2000); // Deposit into savings account
        System.out.println("\nWithdrawing Rs.1500");
        savings.withdraw(1500); // Withdraw from savings account
        System.out.println("\nCalculating Interest");
        savings.calculateInterest(); // Calculate and add interest
        System.out.println("\nUpdated Savings Account:");
        savings.displayAccountDetails();
        System.out.println("\n===== CURRENT ACCOUNT =====");
        current.displayAccountDetails();
        System.out.println("\nDepositing Rs.3000");
        current.deposit(3000);
    }
}

```

```
System.out.println("\nWithdrawing Rs.9000");
current.withdraw(9000);           // Uses overridden withdraw() method
System.out.println("\nUpdated Current Account:");
current.displayAccountDetails();
System.out.println("\n===== TRANSFER =====");
System.out.println(
    "Transferring Rs.1000 from Savings to Current");
savings.transfer(current, 1000);  // Transfer money between accounts
System.out.println("\nSavings Account after transfer:");
savings.displayAccountDetails();
System.out.println("\nCurrent Account after transfer:");
current.displayAccountDetails();
    }
}
```

6. OUTPUT SCREENSHOT

```
C:\Java> javac BankAccountManagement.java
C:\Java> java BankAccountManagement

===== SAVINGS ACCOUNT =====
Account Number : 101
Account Holder : Meghana
Account Type   : Savings
Balance       : 10000.0

Depositing Rs.2000
Amount deposited successfully.

Withdrawing Rs.1500
Amount withdrawn successfully.

Calculating Interest
Interest Rate : 5.0%
Interest Added: 525.0

Updated Savings Account:
Account Number : 101
Account Holder : Meghana
Account Type   : Savings
Balance       : 11025.0

===== CURRENT ACCOUNT =====
Account Number : 102
Account Holder : Rahul
Account Type   : Current
Balance       : 5000.0

Depositing Rs.3000
Amount deposited successfully.
```

The above output represents the terminal output obtained after compiling and executing the Java program.

7. RESULT

Thus, the Bank Account Management System was successfully designed and implemented in Java. The program demonstrates constructors, attributes, methods, object creation, inheritance, method overriding, deposits, withdrawals, interest calculation, overdraft handling, and fund transfer operations. The final balances of the Savings Account and Current Account are displayed successfully.